

SQL SERVER 2022 • AVANZADO

Novedades en el motor y rendimiento en SQL Server 2022

Sesión 1 de 6 • Intelligent Query Processing, Parameter Sensitive Plan, feedback de memoria y paralelismo, Query Store

Cómo vamos a trabajar

- Seis sesiones de 4 horas: aproximadamente un 40 % de exposición y un 60 % de práctica
- Cada bloque teórico termina en un laboratorio guiado con scripts reproducibles
- Una única base de datos para todo el curso: LegacyShop, un sistema de pedidos con deuda técnica real
- Todo el material está en el repositorio de GitHub: laboratorios, datos, soluciones y estas diapositivas
- Regla de oro: no se optimiza nada que no se haya medido antes y después

Lo que necesito de vosotros

Que me contéis qué versión de SQL Server tenéis en producción, qué problemas os encontráis más a menudo y si trabajáis más como DBA, como analistas o como perfiles de seguridad. Con eso ajusto los ejemplos y el ritmo de los laboratorios.

Mapa del curso



Cada sesión se apoya en la anterior: lo que midamos hoy con Query Store lo usaremos para validar todo lo demás.

Entorno de laboratorio

```
# 1. Clonar el repositorio
git clone https://github.com/jorloicono/AF-SQLSERVER-2022-AVANZADO.git
cd AF-SQLSERVER-2022-AVANZADO/setup

# 2. Levantar SQL Server 2022 Developer en Docker
docker compose up -d
docker logs -f sql2022      # esperar a "SQL Server is now ready"

# 3. Crear y poblar LegacyShop (3-6 minutos)
docker exec -it sql2022 /opt/mssql-tools18/bin/sqlcmd \
  -S localhost -U sa -P 'Curso_SQL2022!' -C \
  -i /datos/01_crear_LegacyShop.sql
```

Cliente recomendado: SSMS 21 o superior (Windows) o VS Code con la extensión MSSQL. Ambos muestran planes gráficos.

LegacyShop: nuestro paciente

- Clientes (100.000), Productos (2.000), Pedidos ($\approx 1,2$ M), LineasPedido ($\approx 3,6$ M), LogActividad (montículo sin índices)
- Distribución sesgada a propósito: el cliente 1 es un marketplace con 250.000 pedidos; el resto tiene una decena
- Código heredado: cursores, funciones escalares, variables de tabla gigantes, disparador fila a fila
- Tipos mal alineados: CodigoCliente es VARCHAR pero los procedimientos lo reciben como NVARCHAR
- Datos sensibles sin proteger: IBAN, fecha de nacimiento, email y teléfono en claro

Por qué un sistema "roto"

Porque en la vida real casi nunca diseñáis desde cero. Heredáis sistemas con años de parches, y la habilidad que diferencia a un perfil avanzado es saber priorizar qué arreglar primero con el menor riesgo posible.

01

Novedades de SQL Server 2022

Qué cambia en el motor, qué cambia alrededor, y qué significa el nivel de compatibilidad 160

Cuatro grandes bloques de novedades

1

Procesamiento inteligente de consultas

PSP optimization, DOP feedback, feedback de memoria por percentil y persistente, CE feedback, optimized plan forcing.

2

Seguridad

Ledger, Always Encrypted con enclaves mejorado, TDS 8.0 y TLS 1.3, nuevos roles de servidor granulares, autenticación Microsoft Entra.

3

Híbrido y disponibilidad

Grupos de disponibilidad contenidos, Link a Azure SQL Managed Instance, backup a almacenamiento S3.

4

T-SQL y motor

GENERATE_SERIES, DATE_BUCKET, GREATEST/LEAST, IS DISTINCT FROM, WINDOW, concurrencia de páginas de sistema en tempdb.

Versión del motor frente a nivel de compatibilidad

Llega con la versión 2022

- Mejoras de tempdb (páginas GAM/SGAM)
- Ledger, TDS 8.0, nuevos roles de servidor
- Query Store activado por defecto en bases nuevas
- Grupos de disponibilidad contenidos
- Backup a S3 y compresión con Intel QAT

Requiere compatibilidad 160

- Parameter Sensitive Plan optimization
- DOP feedback (además, activarlo explícitamente)
- Feedback de memoria por percentil
- CE feedback
- Parte de las nuevas funciones de T-SQL

Comprobar y preparar una base de datos para 2022

```
-- ¿En qué nivel está cada base de datos?
SELECT name, compatibility_level, is_query_store_on
FROM sys.databases;

-- Configuración de ámbito de base de datos relevante
SELECT name, value, is_value_default
FROM sys.database_scoped_configurations
WHERE name IN ('PARAMETER_SENSITIVE_PLAN_OPTIMIZATION', 'DOP_FEEDBACK',
               'MEMORY_GRANT_FEEDBACK_PERCENTILE_GRANT', 'MEMORY_GRANT_FEEDBACK_PERSISTENCE',
               'CE_FEEDBACK', 'OPTIMIZED_PLAN_FORCING');

-- Subir de nivel (tras capturar la línea base con Query Store)
ALTER DATABASE LegacyShop SET COMPATIBILITY_LEVEL = 160;
```

Nunca subáis el nivel sin Query Store activo y con una línea base capturada.

02

Parameter Sensitive Plan optimization

El problema del parameter sniffing y cómo SQL Server 2022 guarda varios planes para una misma consulta

Parameter sniffing: un buen comportamiento que sale mal

- Al compilar un procedimiento, el optimizador "olfatea" el valor real del parámetro y estima filas con el histograma
- El plan resultante se guarda en caché y se reutiliza para cualquier valor posterior
- Con datos uniformes es ideal: se ahorra CPU de compilación
- Con datos sesgados es un problema: el plan óptimo para 10 filas es desastroso para 250.000, y viceversa
- Síntoma típico: "ayer iba bien y hoy va lento", y se arregla solo al reiniciar o recompilar

En LegacyShop

usp_PedidosPorCliente con
@ClienteID = 42 devuelve unas 10
filas: Index Seek + Key Lookup,
perfecto. Con @ClienteID = 1
devuelve 250.000: lo correcto sería un
Scan. Quien compile primero decide
el plan para todos.

Soluciones clásicas frente a PSP

Antes de 2022

- OPTION (RECOMPILE): plan perfecto, CPU de compilación en cada ejecución
- OPTIMIZE FOR (@p = ...) o UNKNOWN: plan fijo, bueno para unos y malo para otros
- Copiar el parámetro a una variable local: equivale a UNKNOWN
- Procedimientos distintos por rango, o SQL dinámico
- Plan guides o forzar plan en Query Store

Con PSP en 2022

- El optimizador detecta el sesgo en el histograma
- Crea un "dispatcher" y hasta 3 variantes de plan
- Cada ejecución se enruta según la cardinalidad estimada
- Sin tocar el código de la aplicación
- Visible en Query Store como variantes de la consulta

Cómo funciona PSP por dentro

1

Compilación

Predicado de igualdad con parámetro sobre columna con estadísticas

2

Detección de sesgo

El histograma revela valores muy frecuentes frente a valores raros

3

Dispatcher

Plan raíz que define fronteras de cardinalidad (por defecto 100 y 1.000.000 filas)

4

Variantes

Hasta 3 planes: baja, media y alta cardinalidad

5

Ejecución

Cada valor se enruta a su variante y reutiliza su plan en caché

En el XML del plan aparece `OPTION (PLAN PER VALUE(...))` con el `QueryVariantID` y el `predicate_range`.

Diagnosticar PSP con Query Store y Extended Events

```
-- Variantes generadas por PSP
SELECT qv.parent_query_id, qv.query_variant_query_id, qv.dispatcher_plan_id,
       qt.query_sql_text, p.plan_id, rs.count_executions, rs.avg_duration/1000.0 AS avg_ms
FROM sys.query_store_query_variant qv
JOIN sys.query_store_query q      ON q.query_id = qv.query_variant_query_id
JOIN sys.query_store_query_text qt ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p      ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id;

-- ¿Por qué NO se ha aplicado PSP a una consulta?
CREATE EVENT SESSION psp_diag ON SERVER
ADD EVENT sqlserver.parameter_sensitive_plan_optimization_skipped_reason,
ADD EVENT sqlserver.parameter_sensitive_plan_optimization
ADD TARGET package0.ring_buffer;
```

Limitaciones que conviene conocer

- Solo predicados de igualdad, y como máximo 3 predicados considerados por consulta
- Requiere compatibilidad 160; se desactiva con `PARAMETER_SENSITIVE_PLAN_OPTIMIZATION = OFF`
- No actúa si el código ya usa `RECOMPILE`, `OPTIMIZE FOR` o copia el parámetro a una variable local
- Tres cubos pueden no bastar: dentro de "media cardinalidad" puede seguir habiendo sniffing
- Más planes en caché: vigilar el tamaño de la caché y la limpieza tras cambios de esquema

Consejo práctico

PSP no sustituye a un buen diseño de índices. Si la variante de alta cardinalidad sigue siendo lenta, el problema ya no es el sniffing: es el acceso a datos, y eso lo resolveremos en la sesión 4.

Lab 1 • Parameter sniffing y PSP

- Reproducir el parameter sniffing en compatibilidad 150 con usp_PedidosPorCliente
- Medir con STATISTICS IO/TIME y leer los dos planes posibles
- Subir a 160 y observar el dispatcher y las variantes
- Localizar las variantes en sys.query_store_query_variant
- Averiguar con Extended Events por qué PSP no actúa sobre usp_PedidosPorCliente_Legacy

40 min

labs/Lab01_PSP.md

03

Feedback: memoria, paralelismo y Query Store

Un optimizador que aprende de sus ejecuciones anteriores

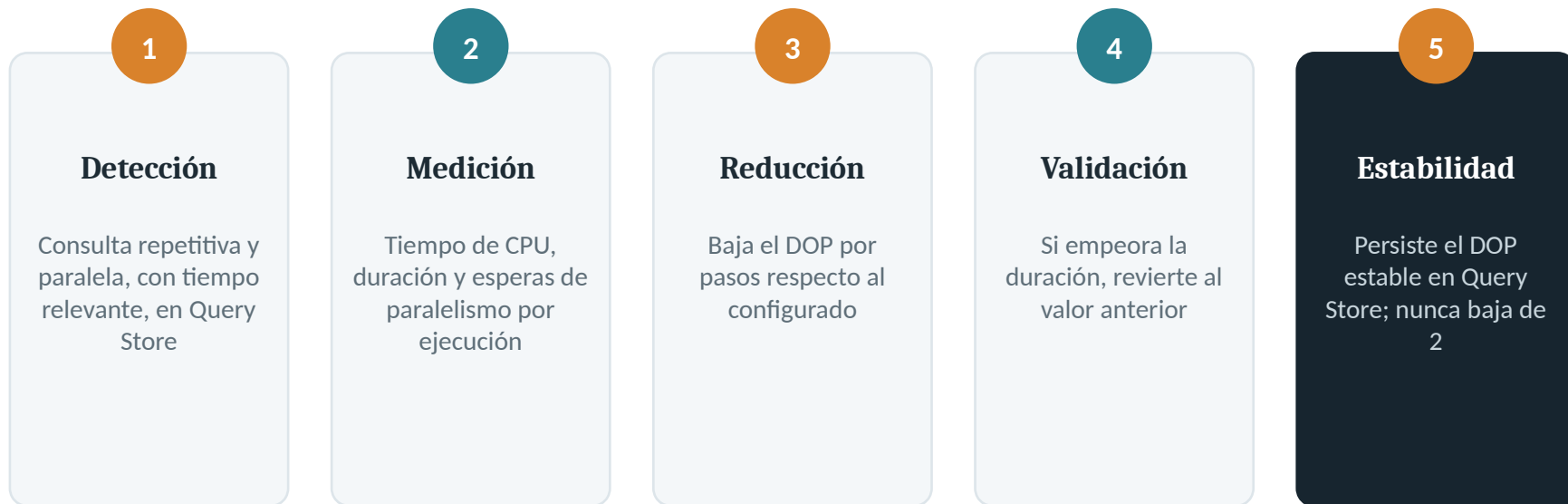
Memory Grant Feedback: percentil y persistencia

- Cada consulta con ordenaciones o hash joins pide una concesión de memoria antes de ejecutarse
- Concesión corta: spill a tempdb (lento). Concesión excesiva: memoria bloqueada y esperas RESOURCE_SEMAPHORE
- Desde 2017-2019 el motor ajustaba la concesión según la última ejecución, lo que oscilaba con cargas variables
- 2022, percentil: ajusta según el historial reciente de ejecuciones, no solo la última
- 2022, persistencia: el ajuste se guarda en Query Store y sobrevive a reinicios y expulsiones de caché

Dónde verlo

En el plan real: atributos `IsMemoryGrantFeedbackAdjusted` y `LastRequestedMemory` dentro de `MemoryGrantInfo`. En Query Store: `sys.query_store_plan_feedback` con `feature_desc = Memory Grant Feedback`.

DOP feedback: paralelismo que se autoajusta



Desactivado por defecto en SQL Server 2022: ALTER DATABASE SCOPED CONFIGURATION SET DOP_FEEDBACK = ON;

Query Store en SQL Server 2022

1

Activado por defecto

Toda base de datos nueva nace con Query Store en READ_WRITE y captura AUTO. Las bases restauradas o migradas conservan su configuración.

2

Query Store hints

Aplicar sugerencias (MAXDOP, RECOMPILE, USE HINT...) a una consulta sin tocar el código, con `sys.sp_query_store_set_hints`.

3

Memoria del feedback

PSP, memoria, DOP y CE feedback guardan su aprendizaje aquí. Sin Query Store, 2022 pierde gran parte de su inteligencia.

4

Réplicas secundarias

Captura en réplicas de lectura de grupos de disponibilidad: en 2022 llega en preview y hay que habilitarla expresamente.

Query Store hints y vista de feedback

```
-- Localizar la consulta problemática
SELECT q.query_id, qt.query_sql_text
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON qt.query_text_id = q.query_text_id
WHERE qt.query_sql_text LIKE N'%LineasPedido%GROUP BY%';

-- Aplicar una sugerencia sin tocar el código
EXEC sys.sp_query_store_set_hints @query_id = 42,
    @query_hints = N'OPTION (MAXDOP 2, USE HINT(''ENABLE_QUERY_OPTIMIZER_HOTFIXES''))';
SELECT * FROM sys.query_store_query_hints;
EXEC sys.sp_query_store_clear_hints @query_id = 42;

-- Qué ha aprendido el motor
SELECT plan_id, feature_desc, feedback_data, state_desc
FROM sys.query_store_plan_feedback;
```

Lab 2 · Feedback de memoria, DOP y Query Store

- Provocar concesiones de memoria erróneas y observar el ajuste por percentil
- Comprobar que el ajuste persiste tras limpiar la caché de planes
- Activar DOP_FEEDBACK y ejecutar una carga repetitiva paralela
- Leer sys.query_store_plan_feedback e interpretar su estado
- Aplicar y retirar una Query Store hint sobre una consulta de informe

45 min

labs/
Lab02_Feedback_QueryStore.
md

Cierre de la sesión 1

- Subir a compatibilidad 160 es una decisión consciente, con Query Store y línea base previa
- PSP resuelve gran parte del parameter sniffing en predicados de igualdad sin tocar código
- El feedback de memoria y de DOP corrige estimaciones y paralelismo, y ahora persiste
- Query Store es la memoria del motor y nuestra herramienta de medición para todo el curso
- Próxima sesión: cómo habla el motor con el disco, tempdb y wait stats de I/O

Para casa

Lanzad en vuestro entorno real la consulta de `sys.databases` y apuntad cuántas bases de datos están por debajo de 160 y cuántas tienen Query Store apagado. Lo comentamos al empezar la próxima sesión.